

Research Task 0: Understanding the BigBird Model

Dalah Alhashlamon

October 9, 2026

1 Introduction

Transformer models such as BERT are very good at understanding text, but they have a big problem: they cannot read long texts easily. The reason is the self-attention mechanism, where every word looks at every other word. If a text has n words, the model must do about $n \times n$ comparisons. So when the text becomes twice as long, the cost becomes about four times larger.

BigBird is a model proposed by Google researchers in the paper “*Big Bird: Transformers for Longer Sequences*” [1]. Its main idea is simple: instead of letting every word look at every other word, let each word look at only a small, carefully chosen set of words. This makes the cost grow in a straight line with the text length instead of squared. In this report I explain the motivation behind BigBird, how it works, how it compares with BERT and Longformer, where it is useful, and what its strengths and weaknesses are. I also used a YouTube video to help me understand the main ideas [2].

2 Motivation: The Problem with Full Attention

In the original Transformer, self-attention builds a table (matrix) of size $n \times n$ that stores how much each word should pay attention to every other word. This causes two problems:

- **Memory:** the table grows quadratically, $O(n^2)$, so long inputs quickly run out of GPU memory.
- **Time:** computing the whole table is also $O(n^2)$.

Because of this, BERT-style models usually accept only 512 tokens. A long document, such as a research paper, a legal contract, or a patent, has to be cut into pieces. Cutting the text loses information that connects the pieces, which hurts tasks like question answering and summarization where the answer may be spread across the whole document.

The goal of BigBird is therefore: **keep the power of full attention, but make the cost linear in the sequence length**. The authors report that BigBird can handle sequences about 8 times longer than what was possible before on similar hardware.

3 How BigBird Works

3.1 Attention as a Graph

The authors look at attention as a graph. Each token is a node, and an edge means “this token can look at that token”. Full attention is a complete graph, where every node is connected to every other node. BigBird uses a *sparse* graph with far fewer edges, but designed so that information can still travel quickly between any two tokens.

3.2 The Three Types of Attention

BigBird combines three simple patterns (see Figure 1):

1. **Window (local) attention.** Each token looks at a few tokens to its left and right. Most of the meaning of a word comes from the words near it, so this keeps local context.
2. **Global tokens.** A small number of special tokens look at *all* tokens, and all tokens look at them. They work like “meeting points” that collect and share information across the whole text. In BigBird there are two versions:
 - **ITC** (Internal Transformer Construction): some tokens that already exist in the text, such as [CLS], are made global.
 - **ETC** (Extended Transformer Construction): extra new global tokens are added to the input.
3. **Random attention.** Each token also looks at a few randomly chosen tokens. Random links act as shortcuts. In a random graph, any two nodes are connected by a short path (the path length grows only like $\log n$), so information does not need many steps to travel from one end of the text to the other.

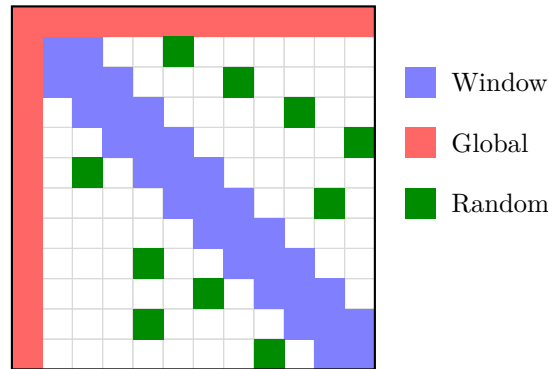


Figure 1: A simple picture of BigBird attention for 12 tokens. Colored cells are the pairs of tokens that can attend to each other: window (blue), global (red), random (green). White cells are skipped. Full attention would color every cell.

3.3 Block-Sparse Implementation

Picking single random tokens is slow on GPUs, because GPUs are fast when they work on big chunks of data at once. To fix this, BigBird groups tokens into **blocks** (for example 64 tokens per block) and applies the window, global, and random patterns to blocks instead of single tokens. This turns the sparse attention into regular matrix multiplications that GPUs and TPUs handle efficiently. This is what is meant by *block-sparse* attention.

3.4 Why It Still Works: Theory in Simple Words

A sparse model could lose power compared to full attention, so the authors prove two things:

- BigBird is a **universal approximator** of sequence functions, meaning it can in principle represent any such function, as full attention can.
- BigBird is **Turing complete**, meaning it can in principle perform any computation, as full attention can.

The analysis also shows that having a few global tokens (like [CLS]) is important for these properties. The authors also point out a trade-off: there are some tasks that full attention can solve with very few layers, but a sparse model needs many more layers to solve. So sparsity is not completely free.

4 Computational Complexity: BERT vs. Longformer vs. BigBird

Model	Attention pattern	Complexity	Typical max length
BERT	Full (every token sees every token)	$O(n^2)$	512
Longformer	Sliding window + global tokens	$O(n \cdot w)$	4096
BigBird	Window + global + random	$O(n(w + g + r))$	4096

Table 1: Attention cost of the three models. n = sequence length, w = window size, g = number of global tokens, r = number of random tokens per token.

Small example. Take $n = 4096$ tokens. Full attention needs $4096^2 \approx 16.8$ million token pairs. If BigBird lets each token see about 8 blocks of 64 tokens (that is 512 tokens), it needs only about $4096 \times 512 \approx 2.1$ million pairs, which is about 8 times fewer. (These numbers are only an illustration; real settings can differ.)

Main points of the comparison:

- BERT is $O(n^2)$, so it becomes very expensive for long inputs.
- Longformer and BigBird are both *linear* in n when w , g , and r are fixed numbers.
- BigBird differs from Longformer because it **adds random attention**, it comes with **theoretical proofs**, and it uses a **block-sparse** design that is friendly to hardware.
- In the paper’s experiments, using window, global, and random attention together works better than using any one of them alone.

5 Applications for Long Documents

BigBird is useful whenever the input is longer than about 512 tokens:

- **Question answering over long texts** (for example HotpotQA, Natural Questions, TriviaQA, WikiHop), where the evidence is spread over many paragraphs.
- **Summarization** of long documents such as scientific papers (arXiv, PubMed) and patents. In the paper, BigBird is used as the encoder together with a Pegasus-style decoder.
- **Long document classification**, such as classifying legal, medical, scientific, or patent documents.
- **Genomics**: DNA is a very long sequence, so the authors also apply BigBird to tasks such as promoter region prediction and chromatin profile prediction.

6 Advantages, Limitations, and Performance

6.1 Advantages

- Memory and time grow linearly with the sequence length.
- It can read much longer inputs (about 8 times longer) without cutting them.

- It has theoretical proofs that it keeps the power of full attention.
- It can be pretrained like BERT-style models, and pretrained versions are available (for example in the Hugging Face library).

6.2 Limitations

- For short texts, it gives no real benefit and adds extra complexity (the Hugging Face version switches back to full attention for short inputs).
- The block-sparse implementation is more complicated than normal attention, and inputs must be padded to fit the block size.
- Random attention is harder for hardware to run fast, so the real speed-up is smaller than the theory suggests.
- Some tasks need many more layers with sparse attention than with full attention.
- Its limit of about 4096 tokens is still short for very long inputs such as whole books.
- It is mainly an encoder, so for text generation it must be combined with a decoder.

6.3 Performance

In the paper, the longer context helps BigBird improve results on question answering and summarization compared with models limited to shorter inputs, and it also gives strong results on the genomics tasks [1]. For exact scores, please see the result tables in the original paper.

7 Conclusion

BigBird solves the main weakness of standard Transformers, the quadratic cost of attention, by combining three simple ideas: local window attention, a few global tokens, and random attention. This gives linear cost, allows inputs about eight times longer than before, and still keeps the theoretical power of full attention. It is a good choice for tasks with long inputs such as long-document question answering, summarization, classification, and genomics, although it brings little benefit for short texts and has a more complex implementation.

References

- [1] Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, Amr Ahmed’ *Big Bird: Transformers for Longer Sequences*, 2020.
- [2] [Yannic Kilcher]. “[Big Bird: Transformers for Longer Sequences (Paper Explained)].” YouTube, [2020]. [<https://www.youtube.com/watch?v=WVPE62Gk3EM>]